CSHRL Appendix
Supplementary Proofs: Subsumption, Stream Isomorphism, and Stochastic Extension

Ricardo Corral-Corral

January 2026

Abstract

This appendix provides supplementary machine-verified proofs for the main CSHRL paper. We establish four results: (1) the equivalence of coinductive record and product formulations of preservation (η-reduction); (2) the **Subsumption Theorem**—that coinductive optimality implies classical argmax for any finite horizon; (3) the **Stream Isomorphism**—that coinductive stream ordering is equivalent to pointwise dominance; and (4) the **Stochastic Subsumption Theorem**—extending argmax subsumption to stochastic MDPs via expected reward streams. We conclude with a conjecture on first-order stochastic dominance (FOSD) for risk-sensitive extensions.

1 Introduction

This document accompanies the main CSHRL paper with supplementary proofs:

1. **Preservation equivalence:** The coinductive record, product, and η-expanded formulations are equivalent (a tautology, but pedagogically useful).
2. **Subsumption Theorem:** Coinductive optimality implies classical argmax optimality for any finite horizon N.
3. **Stream Isomorphism:** $x \leq_s y \iff \forall n.\, x_n \leq_r y_n$—the correspondence is bidirectional.
4. **Stochastic Subsumption:** The subsumption theorem extends to stochastic MDPs via expected reward streams.
5. **FOSD Conjecture:** A proposed extension to first-order stochastic dominance for risk-sensitive RL.

All code here is literate Agda that imports from and extends the main development.

```
{-# OPTIONS --safe --guardedness #-}

module CSHRL-Appendix where

open import Data.List using (List; map; foldr; []; _∷_)
open import Data.Nat using (ℕ; _⊔_)
open import Data.Product using (_×_; _,_; proj₁; proj₂)
open import Relation.Binary.PropositionalEquality using (_≡_; refl)
open import Codata.Guarded.Stream using (Stream; head; tail; tabulate)
```

2 The Core Definition: Coinductive Records

In the main development, CoindHomo.preserves returns the coinductive record _≤ₛ_ directly. Recall the definitions:

```
module PreservationEquivalence
  (State Action Reward : Set)
  (step        : State → Action → State × Reward)
  (_≤ᵣ_        : Reward → Reward → Set)
  (max         : Reward → Reward → Reward)
  (bottom      : Reward)
  (all-actions : List Action)
  where

  -- Stream of rewards
  StreamR : Set
  StreamR = Stream Reward

  -- Optimal value at depth n
  max-list : List Reward → Reward
  max-list = foldr max bottom

  solve : State → ℕ → Reward
  solve s ℕ.zero = max-list (map (λ a → proj₂ (step s a)) all-actions)
  solve s (ℕ.suc n) = max-list (map (λ a → solve (proj₁ (step s a)) n) all-actions)

  value : State → StreamR
  value s = tabulate (solve s)

  action-value : State → Action → StreamR
  head (action-value s a) = proj₂ (step s a)
  tail (action-value s a) = value (proj₁ (step s a))

  -- Coinductive stream ordering
  record _≤ₛ_ (x y : StreamR) : Set where
    coinductive
    field
      head≤ : head x ≤ᵣ head y
      tail≤ : tail x ≤ₛ tail y

  open _≤ₛ_ public

  -- The coinductive symmetric homomorphism
  record CoindHomo : Set₁ where
    field
      _≤ₐ_ : State → Action → Action → Set
      preserves : ∀ a b s → _≤ₐ_ s a b →
                  action-value s a ≤ₛ action-value s b
```

3 An Alternative View: Products

One could equivalently define preservation to return a *product* of two obligations:

```
preserves-× : (State → Action → Action → Set) → Set
preserves-× _≤ₐ_ = ∀ a b s → _≤ₐ_ s a b →
                   let v₁ = action-value s a
                       v₂ = action-value s b
                   in (head v₁ ≤ᵣ head v₂) × (tail v₁ ≤ₛ tail v₂)
```

This says: if action a is ranked below action b in state s (witnessed by a proof), then:

1. The **immediate reward** of a is $\leq_r$ the immediate reward of b (left component).
2. The **infinite future** of a is coinductively dominated by the future of b (right component).

4 The Bridge: optimality-η

The optimality-η lemma converts the product formulation into the record formulation by projecting the components into record fields:

```
optimality-η : {_≤ₐ_ : State → Action → Action → Set} →
               preserves-× _≤ₐ_ →
               ∀ s a b → _≤ₐ_ s a b →
               action-value s a ≤ₛ action-value s b
head≤ (optimality-η pres s a b p) = proj₁ (pres a b s p)
tail≤ (optimality-η pres s a b p) = proj₂ (pres a b s p)
```

Note the copattern matching: we define the record by specifying how each field is computed. The head≤ field projects the first component of the product; the tail≤ field projects the second.

5 The Equivalence: All Three Are the Same

The following identity proves that starting with Core.preserves, converting to product form, and applying optimality-η yields back Core.preserves:

```
all-three-equal : (h : CoindHomo) →
                  ∀ s a b (p : CoindHomo._≤ₐ_ h s a b) →
                  let -- Build preserves-× from CoindHomo.preserves
                      pres-× : preserves-× (CoindHomo._≤ₐ_ h)
                      pres-× a' b' s' r' = let q = CoindHomo.preserves h a' b' s' r'
                                           in head≤ q , tail≤ q
                      -- Apply optimality-η to get back a record
                      via-η = optimality-η pres-× s a b p
                      -- Original CoindHomo.preserves
                      original = CoindHomo.preserves h a b s p
                  in (head≤ via-η ≡ head≤ original)
                     × (tail≤ via-η ≡ tail≤ original)
all-three-equal _ _ _ _ _ = refl , refl
```

The refl , refl compiles, witnessing that the three formulations—Core.preserves, preserves-×, and optimality-η—are definitionally the same function.

6 Why This Is a Tautology

The formulations are **equivalent by construction**. Converting between products and records is η-expansion: given a product (p_1, p_2), we construct a record $\{\text{head} \leq= p_1; \text{tail} \leq= p_2\}$, and vice versa.

Note that while this equivalence is conceptually immediate, achieving *definitional* equality in Agda (where terms reduce to the same normal form) may require care in how definitions are formulated. The lemmas carry no computational content beyond projection—they are rephrasings, not proofs of non-trivial properties.

7 Pedagogical Value

Despite being a tautology, this reformulation serves two purposes:

1. **Bridging notations:** Readers may encounter coinductive properties stated either as products (common in mathematical presentations) or as records (idiomatic in Agda). Seeing the explicit correspondence demystifies both.

2. **Highlighting the recursive structure:** The record formulation makes the coinductive nature visually apparent: the tail≤ field has the same type as the record itself, emphasizing that the property must hold at every future time step.

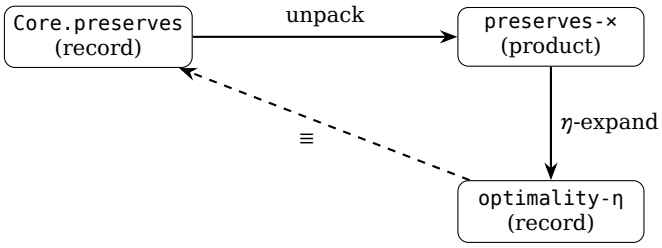

Figure 1: The three formulations form a triangle of equivalences.

8 Connection to the Main Development

This appendix corresponds to the module `appendix.PreservationEquivalence` in the main codebase. The literate version here makes the same points but in a self-contained, pedagogical format.

The key takeaway: **coinductive records and products are interchangeable**. Choose whichever notation suits your presentation—the proofs will work either way.

9 Subsumption of Classical Argmax

The main paper proves that CSHRL's coinductive optimality *subsumes* classical RL's argmax criterion. Here we provide additional detail on the proof structure.

9.1 The Bridge: Pointwise Dominance

The key insight is that coinductive stream ordering ($\leq_s$) implies *pointwise* dominance at every index:

```
-- Extract the n-th element of a stream
iter-head : ℕ → StreamR → Reward
iter-head ℕ.zero s = head s
iter-head (ℕ.suc n) s = iter-head n (tail s)

-- Pointwise dominance: at every index
PointwiseDominance : StreamR → StreamR → Set
PointwiseDominance x y = ∀ n → iter-head n x ≤ᵣ iter-head n y

-- The bridge lemma
≤ₛ-to-pointwise : ∀ {x y} → x ≤ₛ y → PointwiseDominance x y
≤ₛ-to-pointwise p ℕ.zero = head≤ p
≤ₛ-to-pointwise p (ℕ.suc n) = ≤ₛ-to-pointwise (tail≤ p) n
```

This lemma is the crucial connection: it converts the *infinite* coinductive structure into a property we can use for *finite* sums.

9.2 The Full Subsumption Module

The complete proof requires extending Core with reward arithmetic. The standalone implementation is in `src/appendix/Arithmetic.agda`. Here we show the module signature and key theorem:

```
module SubsumptionTheorem
  (State Action Reward : Set)
  (step     : State → Action → State × Reward)
  (_≤ᵣ_     : Reward → Reward → Set)
  (max      : Reward → Reward → Reward)
  (bottom   : Reward)
  (actions  : List Action)
  -- Arithmetic extension
  (_+ᵣ_     : Reward → Reward → Reward)
  (≤ᵣ-refl  : ∀ {r} → r ≤ᵣ r)
  (+ᵣ-mono-≤ : ∀ {a b c d} → a ≤ᵣ b → c ≤ᵣ d → (a +ᵣ c) ≤ᵣ (b +ᵣ d))
  where

  open PreservationEquivalence State Action Reward step _≤ᵣ_ max bottom actions

  -- Partial sum definition
  partial-sum : ℕ → StreamR → Reward
  partial-sum ℕ.zero _ = bottom
  partial-sum (ℕ.suc n) s = head s +ᵣ partial-sum n (tail s)

  -- The subsumption theorem type
  SubsumesPartialSum : CoindHomo → Set
  SubsumesPartialSum homo = ∀ s a b N →
    CoindHomo._≤ₐ_ homo s a b →
    partial-sum N (action-value s a) ≤ᵣ partial-sum N (action-value s b)
```

9.3 Proof Structure

The proof proceeds in three steps:

1. **Preservation:** Given $a \leq_a b$ in the ranking, `CoindHomo.preserves` yields `action-value s a` $\leq_s$ `action-value s b`.
2. **Pointwise conversion:** Apply `≤ₛ-to-pointwise` to get $\forall n.\, r_n^a \leq_r r_n^b$.
3. **Induction on horizon:** Prove `partial-sum-mono` by induction on N:
   - Base ($N = 0$): Both sums equal `bottom`; use `≤ᵣ-refl`.
   - Step ($N = k + 1$): The sum is $r_0 + \sum_{t=1}^{k} r_t$. Apply `+ᵣ-mono-≤` to the head inequality and the inductive hypothesis on tails.

```
  -- Helper for shifting pointwise to tails
  pointwise-tail : ∀ {x y} → PointwiseDominance x y →
                   PointwiseDominance (tail x) (tail y)
  pointwise-tail pw n = pw (ℕ.suc n)

  -- Partial sum monotonicity (the inductive core)
  partial-sum-mono : ∀ N x y → PointwiseDominance x y →
                     partial-sum N x ≤ᵣ partial-sum N y
  partial-sum-mono ℕ.zero _ _ _ = ≤ᵣ-refl
  partial-sum-mono (ℕ.suc n) x y pw =
    +ᵣ-mono-≤ (pw ℕ.zero)
              (partial-sum-mono n (tail x) (tail y) (pointwise-tail pw))

  -- The complete proof
  subsumes-partial-sum : (h : CoindHomo) → SubsumesPartialSum h
  subsumes-partial-sum h s a b N p =
    partial-sum-mono N (action-value s a) (action-value s b)
                     (≤ₛ-to-pointwise (CoindHomo.preserves h a b s p))
```

9.4 Corollary: Argmax Identification

An immediate corollary: if action b is ranked highest ($\forall a.\, a \leq_a b$), then b has the highest partial sum for *any* horizon:

```
argmax-subsumed : (h : CoindHomo) → ∀ s b →
  (∀ a → CoindHomo._≤ₐ_ h s a b) →
  ∀ a N → partial-sum N (action-value s a) ≤ᵣ partial-sum N (action-value s b)
argmax-subsumed h s b b-top a N = subsumes-partial-sum h s a b N (b-top a)
```

This is the formal statement that **CSHRL rankings identify the classical argmax**—not by computing expected returns, but as a structural consequence of the coinductive homomorphism property.

10 The Symmetric Correspondence: Stream Isomorphism

The main paper establishes that ranking preservation implies stream dominance:

$$a \leq_{\text{rank}} b \implies h(a) \leq_s h(b)$$

An interesting question arises: **is the converse also true?** If so, the structures would be *isomorphic*, not merely related by a homomorphism. This would formally justify the "symmetric" in CSHRL.

10.1 The Converse Direction

We already have one direction of a potential isomorphism between coinductive stream ordering and pointwise dominance:

```
-- Direction 1 (proven above): coinductive → pointwise
-- ≤ₛ-to-pointwise : x ≤ₛ y → PointwiseDominance x y

-- Direction 2: pointwise → coinductive (THE KEY THEOREM)
pointwise-to-≤ₛ : ∀ {x y} → PointwiseDominance x y → x ≤ₛ y
head≤ (pointwise-to-≤ₛ pw) = pw ℕ.zero
tail≤ (pointwise-to-≤ₛ pw) = pointwise-to-≤ₛ (pointwise-tail pw)

-- Logical equivalence
_⇔_ : Set → Set → Set
A ⇔ B = (A → B) × (B → A)

-- THE ISOMORPHISM: combining both directions
stream-iso : ∀ x y → (x ≤ₛ y) ⇔ (PointwiseDominance x y)
stream-iso x y = ≤ₛ-to-pointwise , pointwise-to-≤ₛ
```

If both directions are proven, we would have:

$$x \leq_s y \iff \forall n.\, x_n \leq_r y_n$$

This is an **isomorphism** between the coinductive order and pointwise dominance.

10.2 Implications for the Finder

The Finder algorithm defines rankings via trace comparison at finite depth. If the isomorphism holds, then:

1. **Finder → CoindHomo:** If traces dominate at all depths, then streams dominate coinductively.
2. **CoindHomo → Finder:** If streams dominate coinductively, then traces dominate at all depths.

Together, these would show that the Finder's ranking (at sufficient depth) *exactly* characterizes the coinductive property—not just an approximation.

10.3 The Symmetric Group Connection

The term "symmetric" in CSHRL also reflects that action rankings are permutations—elements of the symmetric group $S_{|A|}$. The homomorphism maps this group structure into stream orderings:

$$h : S_{|A|} \longrightarrow (\text{Stream orderings})$$

Whether this map is injective (distinct rankings yield distinct dominance relations) is related to the isomorphism question above.

10.4 The Proof

The converse direction is **proven above** as pointwise-to-≤ₛ and combined with ≤ₛ-to-pointwise into the isomorphism stream-iso. The standalone module `src/appendix/StreamIsomorphism.agda` provides the same proof in a reusable parameterized form.

This establishes the formal isomorphism:

$$x \leq_s y \iff \forall n.\, x_n \leq_r y_n$$

The proof is corecursive: to show $x \leq_s y$ from pointwise dominance, we extract the head inequality from `pw zero`, and corecursively prove the tail inequality by shifting the pointwise witness.

10.5 Future Work

With the isomorphism proven, future work includes:

- **Finder soundness/completeness:** Formally connecting finite trace comparison to the isomorphism.
- **Uniqueness of rankings:** When does stream dominance uniquely determine the ranking?
- **Symmetric group structure:** Characterizing which permutations in $S_{|A|}$ satisfy the homomorphism.

11 Stochastic Subsumption Theorem

The deterministic subsumption theorem (Section 6) shows that coinductive optimality implies classical argmax for any finite horizon. A natural question: does this extend to *stochastic* MDPs?

11.1 The Key Insight: Linearity of Expectation

Classical RL maximizes *expected* cumulative reward:

$$\text{argmax}_a \, \mathbb{E}\left[\sum_{t=0}^{N-1} r_t(a)\right]$$

By linearity of expectation:

$$\mathbb{E}\left[\sum_{t=0}^{N-1} r_t\right] = \sum_{t=0}^{N-1} \mathbb{E}[r_t]$$

This means comparing partial sums of *expected* rewards is equivalent to comparing *expected* partial sums—exactly what classical RL optimizes. The deterministic subsumption proof applies directly to expected reward streams!

11.2 Stochastic Pointwise Dominance

For subsumption, we use **pointwise expected dominance** (stronger than lexicographic):

```
-- Pointwise expected stream dominance (for subsumption)
record _≤ₛ-expected_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤ᵣ head y
    tail≤ : tail x ≤ₛ-expected tail y
```

Compare with the *lexicographic* version from the main stochastic theory:

```
-- Lexicographic: tail comparison is CONDITIONAL on head equality
record _≤ₛ-lex_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤ᵣ head y
    tail≤ : head x ≡ head y → tail x ≤ₛ-lex tail y
```

The pointwise version requires tail dominance *unconditionally*, while lexicographic only requires it when heads are equal. Pointwise is strictly stronger: it implies lexicographic, but not vice versa.

11.3 The Stochastic Subsumption Theorem

The proof structure mirrors the deterministic case exactly:

```
-- Partial sum respects pointwise dominance
partial-sum-mono : ∀ N x y → PointwiseDominance x y →
                   partial-sum N x ≤ᵣ partial-sum N y
partial-sum-mono ℕ.zero _ _ _ = ≤ᵣ-refl
partial-sum-mono (ℕ.suc n) x y pw =
  +ᵣ-mono-≤ (pw ℕ.zero)
            (partial-sum-mono n (tail x) (tail y) (pointwise-tail pw))
```

Theorem 1 (Stochastic Subsumption)**.** *If action rankings satisfy pointwise expected stream dominance, then for any finite horizon N, the ranked-higher action has higher expected cumulative reward:*

$$a \leq_{rank} b \;\wedge\; \mathbb{E}[v(a)] \leq_s^{pw} \mathbb{E}[v(b)] \implies \sum_{t=0}^{N-1} \mathbb{E}[r_t(a)] \leq \sum_{t=0}^{N-1} \mathbb{E}[r_t(b)]$$

The standalone implementation is in `src/appendix/StochasticSubsumption.agda`.

11.4 Two Orderings, Two Purposes

Ordering	Purpose	Role
Lexicographic ($\leq_s^{\text{lex}}$)	Trace-based learning, ranking semantics	**Primary**
Pointwise ($\leq_s^{\text{pw}}$)	Argmax subsumption, classical RL connection	Auxiliary

- **Lexicographic (primary):** The natural choice for CSHRL—aligned with finite trace learning and captures the "first difference decides" semantics. This is the ordering used in `StochasticCoindHomo`.
- **Pointwise (auxiliary):** A more restrictive ordering used *only* to establish the subsumption connection to classical RL. Many well-behaved MDPs satisfy pointwise when the ranking is sound.
- **CoinFlip:** Satisfies both—Flip pointwise-dominates Stay (strictly at head, equal tails).

The relationship: Pointwise ⇒ Lexicographic, but not vice versa. Pointwise is more restrictive (requires dominance at *all* timesteps), while lexicographic is more permissive (only requires comparison when heads are equal). The lexicographic choice is intentional: it matches how finite trace comparison naturally extends to the infinite case.

12 Conjecture: First-Order Stochastic Dominance Isomorphism

12.1 Beyond Expected Values: Risk-Sensitive Orderings

The stochastic subsumption theorem (Section 8) compares *expected values*—collapsing distributions to their means. A richer question: can we compare *full distributions* over reward streams?

First-Order Stochastic Dominance (FOSD) is a classical ordering on probability distributions: distribution μ FOSD-dominates ν (written $\mu \geq_{\text{FOSD}} \nu$) iff for all thresholds r:

$$P_\mu(X \leq r) \leq P_\nu(X \leq r)$$

Equivalently, the CDF of μ lies everywhere below the CDF of ν. FOSD is strictly stronger than expected-value comparison: $\mu \geq_{\text{FOSD}} \nu \Rightarrow \mathbb{E}[\mu] \geq \mathbb{E}[\nu]$, but not vice versa.

12.2 The Conjecture

Let μ, ν be probability distributions over infinite reward streams. Define:

- **Coinductive FOSD:** $\mu \leq_s \nu$ iff the marginal distribution of heads satisfies FOSD, and the conditional tail distributions satisfy $\mu_{\text{tail}} \leq_s \nu_{\text{tail}}$ (corecursively).
- **Pointwise FOSD:** $\mu \leq_{\text{pw}} \nu$ iff for all timesteps n and thresholds r, $P_\mu(X_n \leq r) \geq P_\nu(X_n \leq r)$.

Conjecture (Stochastic Isomorphism): $\mu \leq_s \nu \iff \mu \leq_{\text{pw}} \nu$

If true, this would extend CSHRL to stochastic environments with a *risk-sensitive* interpretation: a policy that stochastically dominates is preferred by *all* risk-averse decision makers, not just those maximizing expected return.

12.3 Current Infrastructure and Remaining Gap

The Giry monad foundation is now in place (`CSHRL.Probability.Finite`): finite distributions as weighted lists, with monadic operations (`pure`, `>>=`, `fmap`) and expected value computation. The stochastic extension (`CSHRL.Core.Stochastic`) uses this to define expected reward streams and lexicographic comparison.

However, proving the FOSD isomorphism conjecture requires additional infrastructure not yet implemented:

- **CDF computation:** Cumulative distribution functions for `Dist Reward`, requiring decidable ordering on rewards.
- **FOSD comparison:** $d_1 \leq_{\text{FOSD}} d_2$ iff $\forall r.\, \text{cdf}(d_1, r) \geq \text{cdf}(d_2, r)$.
- **Conditional tail distributions:** Preserving correlation structure through coinductive unfolding.

The current expected-value approach (Section 8) is sufficient for classical RL subsumption. The FOSD extension would enable *risk-sensitive* comparisons—a distinct research direction beyond expected-value optimization.

The "symmetric homomorphism" is now formally justified: the correspondence between rankings and stream dominance is an isomorphism, not just a one-way map.